



ICTZOID WEB DEVELOPMENT PROGRAM

HTML, CSS & JavaScript 4-Week Plan

A simultaneous learning curriculum covering all three web fundamentals before the React sessions begin — built around weekly integration projects.

HTML

CSS

JavaScript

4 Weeks

Pre-React

Hands-on Builds

Written by

Samuel M. Ortil

ICTZoid · 2026

Goal: Learners build working skills in all three web technologies in parallel, reinforcing each through integration — not isolation. Every week ends with a mini-project that requires all three to work together.

Prerequisite: Master Session 1 — How the Internet Works

What comes next: Week 5 begins React sessions (Course 5)

How This Plan Works

Rather than finishing HTML → CSS → JS in sequence, learners study aligned topics from all three each week. Each trio of topics produces something immediately visible and interactive:

Week	Theme	HTML Topics	CSS Topics	JS Topics	Weekly Build
1	Foundations	Document structure & semantic elements	Selectors, cascade & box model	Variables, data types & functions	Bio page
2	Content & Layout	Links, images, tables & figures	Typography, Flexbox	Arrays & objects	Course card grid
3	Interaction	Accessible forms & ARIA	CSS Grid, responsive design	DOM & events	Responsive landing page
4	Data & Polish	Validation & semantic review	Animations, dark mode & CSS architecture	localStorage, Fetch API & async/await	Interactive data app

Week 1 — Foundations

Every element has structure. Every structure has style. Every script has state.

● HTML (COURSE 1)

- **Lesson 1.1** — HTML document structure: `<!DOCTYPE html>`, `<html>`, `<head>`, `<body>`, meta tags
- **Lesson 1.2** — Semantic elements: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`
- **Lesson 1.3** — Text content: headings (`h1` – `h6`), paragraphs, `<strong>`, `<em>`, `<blockquote>`, `<code>`, `<pre>`

● CSS (COURSE 2 — MODULE 1)

- **Lesson 1.1** — How CSS works: rule syntax, type / class / ID / pseudo-class selectors, combinators, `:is()`
- **Lesson 1.2** — Cascade & specificity: origin, scoring, CSS custom properties (`--color-primary`, `--space-4`, etc.)
- **Lesson 1.3** — The box model: `box-sizing: border-box`, padding, margin, `display` values

● JAVASCRIPT (COURSE 3 — MODULE 1)

- **Lesson 1.1** — Variables & data types: `const` / `let`, all eight types, strict equality (`===`), nullish coalescing (`??`)

- **Lesson 1.2** — Functions: declaration, expression, arrow, default parameters, rest, destructured params, closures

● WEEKLY BUILD — BIO PAGE

Build a personal bio page that ties everything together:

- Valid semantic HTML5 document with correct use of `<header>`, `<main>`, `<section>`, `<footer>`
- CSS custom properties for the full token system (colours, spacing, typography)
- `box-sizing: border-box` applied globally
- JavaScript that reads the page's heading and logs a greeting to the console using a closure-based `greet()` function

Definition of done: Valid HTML (W3C Validator passes), one CSS file using only class selectors, JavaScript in a separate `<script src>` file.

Week 2 — Content & Layout

Content needs context. Layout shapes meaning. Data drives display.

● HTML (COURSE 1)

- **Lesson 2.1** — Links & images: `<a>` with `href` / `target` / `rel`, `<img>` with `alt`, `srcset`, `lazy` loading, `<picture>`
- **Lesson 2.2** — Lists: `<ul>`, `<ol>`, `<dl>` (description lists) and when to use each
- **Lesson 2.3** — Tables: `<table>`, `<thead>`, `<tbody>`, `<th scope>`, `<caption>` for accessible data tables
- **Lesson 2.4** — Figures & media: `<figure>`, `<figcaption>`, `<video>`, `<audio>`, `<iframe>`

● CSS (COURSE 2 — MODULE 1-2)

- **Lesson 1.4** — Typography: `font-family` stacks, `line-height`, `rem` / `em` / `ch` units, `clamp()` for fluid sizing
- **Lesson 2.1** — Flexbox fundamentals: `display: flex`, `justify-content`, `align-items`, `gap`, `flex-grow` / `shrink` / `basis`
- **Lesson 2.2** — Flexbox in practice: nav bar, card wrapping, media card, input-button group, sticky footer

● JAVASCRIPT (COURSE 3 — MODULE 1)

- **Lesson 1.3** — Arrays: `map`, `filter`, `reduce`, `find`, `sort`, chaining, non-mutating add/remove patterns
- **Lesson 1.4** — Objects & destructuring: dot/bracket notation, spread, `Object.keys/values/entries`, destructuring with defaults and renaming

● WEEKLY BUILD — COURSE CARD GRID

Build a course catalog page driven by a JavaScript data array:

- HTML skeleton with semantic `<main>` and `<ul>` as the card container
- JavaScript array of 4–6 course objects (title, description, level, duration); rendered into the DOM using `map` + `innerHTML` fragment pattern from Lesson 2.1 JS
- Flexbox card grid that wraps naturally; each card has equal height with a footer that sticks to the bottom
- Fluid typography using `clamp()` for headings; `rem`-based spacing from the token system

Definition of done: Cards render from a JS data array (no hardcoded HTML cards), layout tested at 375px and 1280px, images have meaningful `alt` text.

Week 3 — Interaction

Forms collect. Grids place. Events connect.

● HTML (COURSE 1)

- **Lesson 3.1** — Forms: `<form>`, `<label>`, `<input>` types, `<textarea>`, `<select>`, `<fieldset>`, `<legend>`, required/pattern attributes
- **Lesson 3.2** — Form accessibility: every input paired with a `<label>`, `aria-describedby` for hints and errors, `aria-invalid`
- **Lesson 3.3** — ARIA: `role`, `aria-label`, `aria-expanded`, `aria-hidden`, landmark roles; when HTML semantics are not enough

● CSS (COURSE 2 — MODULE 2-3)

- **Lesson 2.3** — CSS Grid fundamentals: `grid-template-columns`, `auto-fit` + `minmax`, named areas, `grid-column` / `grid-row` placement
- **Lesson 2.4** — Grid in practice: page layout with named areas, dashboard grid, image mosaic; Grid vs Flexbox rule of thumb
- **Lesson 3.1** — Mobile-first responsive design: base styles for mobile, `min-width` breakpoints, standard breakpoints (640 / 768 / 1024 / 1280)
- **Lesson 3.2** — Responsive navigation: hamburger pattern, `is-open` toggle, `slideDown` animation, accessible `aria-expanded`

● JAVASCRIPT (COURSE 3 — MODULE 2)

- **Lesson 2.1** — DOM manipulation: `querySelector`, `classList`, `textContent`, `dataset`, `createElement`, `DocumentFragment`, XSS-safe patterns
- **Lesson 2.2** — Events: `addEventListener`, event object, `preventDefault`, `stopPropagation`, event delegation with `.closest()`, keyboard events

● WEEKLY BUILD — RESPONSIVE LANDING PAGE

Build a landing page for the ICTzoid program:

- CSS Grid page layout (header / sidebar / main / footer using named areas); fluid card grid with `auto-fit minmax(280px, 1fr)`
- Mobile-first: single-column at 375px, two-column at 768px, three-column at 1024px
- Responsive nav with JavaScript hamburger toggle that sets `aria-expanded` and `is-open` class
- FAQ section using event delegation: one listener on the container toggles each answer and updates `aria-expanded`

Definition of done: Layout passes visual inspection at 375px, 768px, and 1280px; hamburger toggle works; FAQ accordion updates ARIA; no `onclick` attributes — `addEventListener` only.

04

Week 4 — Data & Polish

Validate input. Persist state. Fetch the world. Ship with confidence.

● HTML (COURSE 1)

- **Lesson 4.1** — HTML validation: W3C Validator errors, common mistakes (missing `alt`, duplicate IDs, incorrect nesting)
- **Lesson 4.2** — Advanced semantic elements: `<time datetime>`, `<address>`, `<mark>`, `<details>` + `<summary>`, `<dialog>`, `<template>`
- **Lesson 4.3** — Complete HTML checklist: `lang` attribute, `<meta charset>`, `<meta name="viewport">`, `<meta name="description">`, skip-to-content link

● CSS (COURSE 2 — MODULE 3-4)

- **Lesson 3.3** — Transitions & keyframe animations: `transition` on specific properties, `@keyframes`, `fadeUp`, `spin`, `shimmer`, `prefers-reduced-motion`
- **Lesson 3.4** — Theming & dark mode: `data-theme` attribute, `prefers-color-scheme`, `localStorage` persistence, smooth theme transition
- **Lesson 4.1** — CSS architecture: modern reset, token file, component / layout / page folder structure, `@import` composition

● JAVASCRIPT (COURSE 3 — MODULES 2-3)

- **Lesson 2.3** — Form validation with JavaScript: `validateField()`, `showError()` / `showSuccess()`, validate on blur, disable submit until valid
- **Lesson 2.4** — `localStorage`: `setItem` / `getItem` / `JSON.stringify` / `JSON.parse`, safe `getStorage` / `setStorage` helpers, persistent to-do pattern
- **Lesson 3.1** — Event loop & Promises: call stack, task queue, `Promise` constructor, `.then/.catch/.finally`, `Promise.all`
- **Lesson 3.2** — Async/Await: `async function`, `try/catch/finally`, sequential vs parallel awaits, `for ... of` for async loops
- **Lesson 3.3** — Fetch API & REST: `apiFetch` helper, GET/POST/PATCH/DELETE patterns, UI loading / error / empty states

● WEEKLY BUILD — INTERACTIVE DATA APP

Build a **Movie Search** app (or Courses Dashboard) that integrates all four weeks:

- Semantic HTML with `<dialog>` for the detail modal, `<time>` for dates, valid W3C output
- CSS split into base/components/layout files; `@keyframes fadeUp` on cards; dark/light toggle with `data-theme` + `localStorage`
- JavaScript: search input calls a public API (OMDB or JSONPlaceholder), results render as a card grid, clicking a card opens a `<dialog>` with details, recent searches persist in `localStorage`
- Full UI states: skeleton loader while fetching, error message with retry button, "No results" empty state

Definition of done: W3C HTML validation passes, Lighthouse Performance ≥ 90 , dark mode persists on refresh, all four UI states are reachable, no `var`, no `=`, no `!important`.

Each week follows the same structure:

Day	Activity
Monday	HTML lessons + exercises for the week
Tuesday	CSS lessons + exercises for the week
Wednesday	JavaScript lessons + exercises for the week
Thursday	Integration: start the weekly build
Friday	Complete and submit the weekly build; peer review
Weekend	Self-assessment against checklists; fix feedback

Pre-React Readiness Checklist

Confirm every box before Week 5's React sessions begin.

HTML

- ☐ Writes a valid HTML5 document from memory
- ☐ Uses semantic elements (`<header>`, `<main>`, `<section>`, `<article>`, `<div>`, `<small>`, `<em>`, `<strong>`, `<code>`, `<pre>`, `<code>`, `<code>`, etc.) correctly
- ☐ Every `<img>` is associated with a `alt`
- ☐ Images have meaningful `alt` text; decorative images use `alt=""`
- ☐ ARIA attributes used only where HTML semantics are insufficient
- ☐ W3C Validator returns zero errors on submitted work

CSS

- ☐ `box-sizing: border-box` applied globally in every project
- ☐ CSS custom properties used for all colours, spacing, and typography
- ☐ Selects only with classes — no IDs, no `!important`
- ☐ Builds any layout with Flexbox (components) and CSS Grid (page structure)
- ☐ Mobile-first: base styles for mobile, `min-width` queries for larger screens
- ☐ Transitions use `transform` and `opacity` only; `prefers-reduced-motion` respected
- ☐ Dark/light theme working with `data-theme` + `localStorage`

JavaScript

- ☐ Chooses ``const`` vs ``let`` correctly; never uses ``var``
- ☐ Writes functions in all three syntaxes; knows when to use each
- ☐ Transforms arrays with ``map``/``filter``/``reduce`` without mutation
- ☐ Destructures objects and arrays including nested structures
- ☐ Selects, creates, updates, and removes DOM elements
- ☐ Uses ``addEventListener`` and event delegation; no ``onclick`` attributes
- ☐ Validates a form with JavaScript and shows accessible error messages
- ☐ Reads and writes ``localStorage`` with JSON serialisation
- ☐ Writes ``async/await`` with ``try/catch/finally``
- ☐ Makes GET, POST, PATCH, DELETE requests with ``fetch``
- ☐ Handles loading, error, and empty UI states
- ☐ Debugs using ``console`` methods and Chrome DevTools breakpoints